3.1

Suppose we toss darts randomly at a square dartboard, whose bullseye is at the origin, and whose sides are 2 feet in length. Suppose also that there's a circle inscribed in the square dartboard. The radius of the circle is 1 foot, and it's area is π square feet. If the points that are hit by the darts are uniformly distributed (and we always hit the square), then the number of darts that hit inside the circle should approximately satisfy the equation

$$\frac{\text{number in circle}}{\text{total number of tosses}} = \frac{\pi}{4},$$

since the ratio of the area of the circle to the area of the square is $\pi/4$.

We can use this formula to estimate the value of π with a random number generator:

```
number_in_circle = 0;
for (toss = 0; toss < number_of_tosses; toss++) {
   x = random double between -1 and 1;
   y = random double between -1 and 1;
   distance_squared = x*x + y*y;
   if (distance_squared <= 1) number_in_circle++;
}
pi_estimate = 4*number_in_circle/((double) number_of_tosses);
```

Use OpenMPI to write a parallel program for this method. The processes evenly distribute the tosses, while process 0 sums up the total number of darts which hit inside the circle from processes, calculates and outputs the estimate of the value of π.

INPUT: n – the number of total tosses.

OUTPUT: The estimate of the value of π.

3.2

Implement a parallel **k-Nearest Neighbor (k-NN) search** using MPI. You do **not** need to use an external dataset, process 0 generates all using random numbers and distributes to other processes.

Problem Description

Suppose a dataset contains N points, where each point has D numerical features.

A point can be represented as:

$$[\, X_i \;=\; (x_{i1}, x_{i2}, \ldots, x_{iD})]$$

You are also given a query point:

$$[\, Q \;=\; (q_1, q_2, \ldots, q_D)]$$

Your task is to find the k data points that are closest to the query point.

Use the Euclidean distance from Q to point X_i:

$$\sqrt{\sum_{j=1}^{D} (x_{ij} - q_j)^2}$$

The $\boldsymbol{k}$ points with the smallest distances are the $\boldsymbol{k}$ nearest neighbors.

Parallel Implementation

Your program must use MPI.

Assume the dataset contains N points and the program is executed using p MPI processes.

Step 1: Generate the Query Point and the Data Set

Process 0 should generate a random query point (a vector containing D values) and the dataset containing N random points (each is a vector containing D values).

The query point should then be distributed to all MPI processes using an appropriate MPI collective operation, each MPI process should get its own local portion of the dataset.

Step 2: Compute Local Distances

Each process computes the Euclidean distance between the query point and every point in its local dataset.

Each process should determine its own **local *k* nearest neighbors**.

For every local neighbor, store at least:

- the distance to the query point,
- and an identifier or global index of the data point.

Step 3: Find the Global k Nearest Neighbors

After each process has found its local k nearest neighbors, the local results should be sent to Process 0.

Process 0 should combine the candidate neighbors from all processes and determine the final global k nearest neighbors.

Do **not** send all computed distances to Process 0.

Only the local k best candidates from each process should be communicated.

INPUT:

- N – the number of total tosses
- D – the number of features per point
- k – the number of nearest neighbors

OUTPUT

- The query vector
- The final *k* nearest
- The distance of each neighbor from the query point

Performance Evaluation

Run your program with the same input using:

- 1 process
- 2 processes
- 4 processes
- 8 processes

For each run, record the execution time using the provided head file “time.h”

Calculate the speedup:

$$S = \frac{T_1}{T_p}$$

Where T_1 is the execution time using one process, and T_p is the execution time using p processes.

Also calculate parallel efficiency:

$$E_p = \frac{S_p}{p}$$

Create a table showing:

Processes	Execution Time	Speedup	Efficiency
1		1.00	1.00
2			
4			
8			